



# Mini-projet IDM

Amal El Mosslih  
Adam Rochdi

Année: 2024/2025

# Sommaire

|          |                                                                                              |           |
|----------|----------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                                          | <b>2</b>  |
| <b>2</b> | <b>Description des modèles de processus</b>                                                  | <b>2</b>  |
| 2.1      | Exemple de modèle de processus . . . . .                                                     | 2         |
| <b>3</b> | <b>Définition des métamodèles avec Ecore</b>                                                 | <b>2</b>  |
| 3.1      | SimplePDL . . . . .                                                                          | 2         |
| 3.2      | PetriNet . . . . .                                                                           | 3         |
| <b>4</b> | <b>Définitions des contraintes OCL (implantées en Java)</b>                                  | <b>5</b>  |
| 4.1      | Contraintes pour SimplePDL . . . . .                                                         | 5         |
| 4.2      | Contraintes pour PetriNet . . . . .                                                          | 5         |
| <b>5</b> | <b>Syntaxes concrètes de SimplePDL</b>                                                       | <b>5</b>  |
| 5.1      | Syntaxe concrète textuelle de SimplePDL (avec Xtext) . . . . .                               | 5         |
| 5.2      | Syntaxe concrète graphique de SimplePDL (avec Siruis) . . . . .                              | 6         |
| <b>6</b> | <b>Définition de la transformation modèle à modèle (M2M)</b>                                 | <b>7</b>  |
| 6.1      | Définition d'une transformation SimplePDL vers PetriNet en utilisant EM-<br>F/Java . . . . . | 7         |
| 6.2      | Définition d'une transformation SimplePDL vers PetriNet en utilisant ATL                     | 8         |
| <b>7</b> | <b>Définition de la transformation modèle à texte(M2T) en utilisant Ac-<br/>celeo</b>        | <b>9</b>  |
| 7.1      | Définition d'une transformation PetriNet vers Tina . . . . .                                 | 9         |
| 7.2      | Définition d'une transformation SimplePDL vers LTL . . . . .                                 | 10        |
| <b>8</b> | <b>Validation de la transformation SimplePDL2PetriNet</b>                                    | <b>10</b> |
| <b>9</b> | <b>Conclusion</b>                                                                            | <b>11</b> |

# 1 Introduction

Ce mini-projet vise à mettre en place une chaîne de vérification des modèles de processus SimplePDL, dans le but de s'assurer de leur cohérence et, en particulier, de déterminer si le processus modélisé peut se terminer correctement. Pour ce faire, nous utilisons des techniques de *model-checking* en nous appuyant sur les réseaux de Petri, à travers la boîte à outils Tina. La première étape consiste à transformer un modèle de processus en un réseau de Petri afin d'utiliser les outils de vérification disponibles.

Les travaux pratiques (TP) réalisés tout au long du projet introduisent progressivement les outils nécessaires pour accomplir cette tâche. Chaque TP se concentre sur un aspect spécifique du projet, offrant une première exploration des concepts à travers l'utilisation d'outils tels que Ecore, EMF, et d'autres.

Le travail se fera en suivant ces étapes :

- Définition des méta-modèles avec Ecore pour SimplePDL et PetriNet.
- Spécification des règles de cohérence avec OCL (implantation en Java).
- Manipulation des modèles avec l'infrastructure EMF.
- Définition de transformations de modèle à texte (M2T) avec Acceleo, notamment pour générer des fichiers compatibles avec Tina et exprimer les propriétés en LTL.
- Réalisation de transformations de modèle à modèle (M2M) avec Java et ATL.
- Conception de syntaxes concrètes textuelles via Xtext et graphiques via Sirius.

## 2 Description des modèles de processus

### 2.1 Exemple de modèle de processus

Le modèle illustré dans la figure 1 comprend quatre activités : Conception, RédactionDoc, Programmation et RédactionTests, et quatre dépendances : startToStart, startToFinish, finishToStart et finishToFinish.

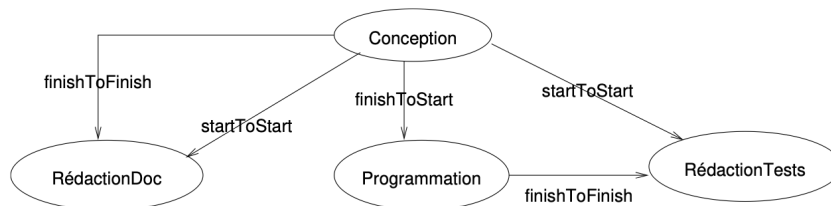


Figure 1: Exemple d'un modèle de processus

## 3 Définition des métamodèles avec Ecore

### 3.1 SimplePDL

On définit le modèle SimplePDL avec quatre activités (Conception, RédactionDoc, Programmation et RédactionTests), quatre dépendances (startToStart, startToFinish, finish-

ToStart et finishToFinish), les ressources, leurs paramètres et guidance (commentaires). Le métamodèle SimplePDL a été étendu pour intégrer la notion de ressource. La classe **Resource** a été introduite, représentant le nombre total de ressources disponibles. La classe intermédiaire **UtilisationRessources** facilite la gestion des ressources nécessaires pour une **WorkDefinition**. Les ressources sont associées à une WorkDefinition via des montants de besoins.

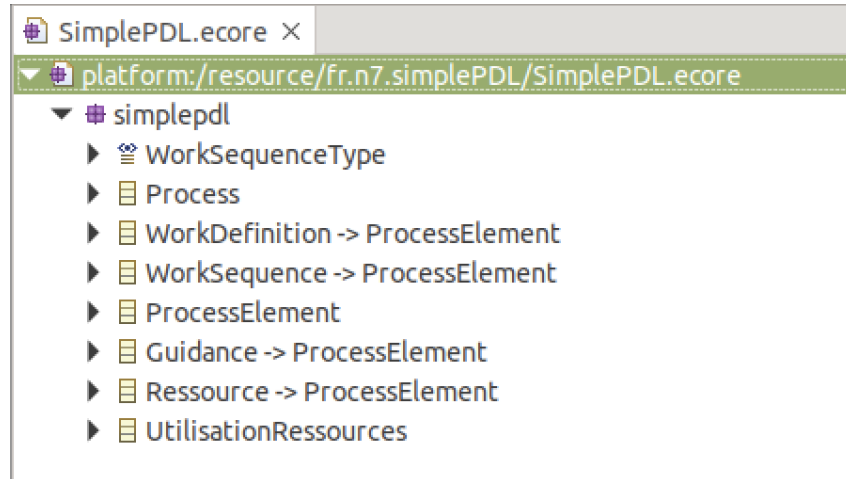


Figure 2: Méta-modèle SimplePDL

On visualise le modèle SimplePDL, et on obtient le méta-modèle dans la figure suivante la figure 3

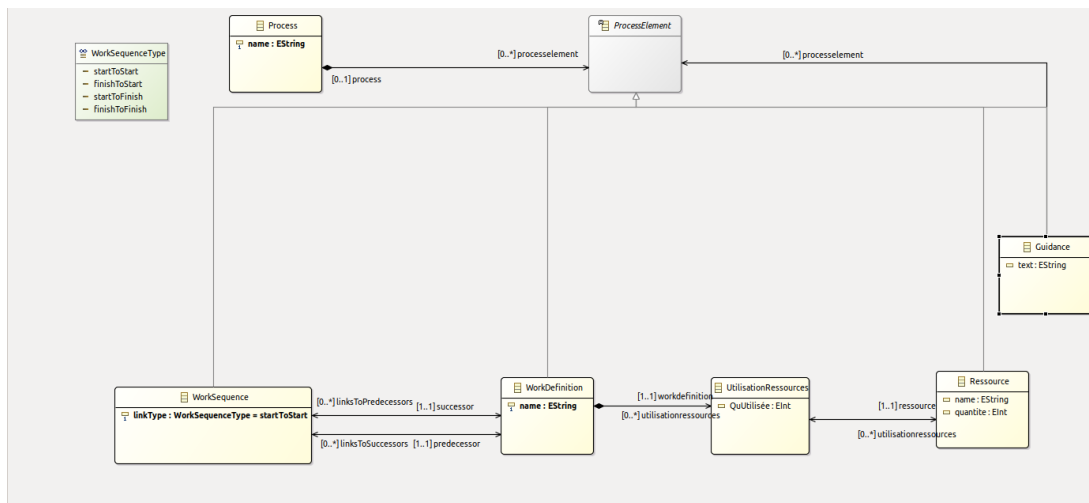


Figure 3: Méta-modèle SimplePDL

## 3.2 PetriNet

Le métamodèle PetriNet formalise les systèmes à éléments discrets en intégrant des **places**, des **transitions** et des arcs qui les connectent. Les arcs peuvent être de type **normal** ou **readArc**, représentant ainsi les flux entre les places et les transitions. Chaque **place** possède un attribut nommé **jetons**, qui indique le nombre initial de jetons présents. Ce modèle permet de décrire de manière formelle les interactions entre les différentes

composantes d'un système à éléments discrets, en saisissant les transitions d'état et les flux de jetons entre les places et les transitions.

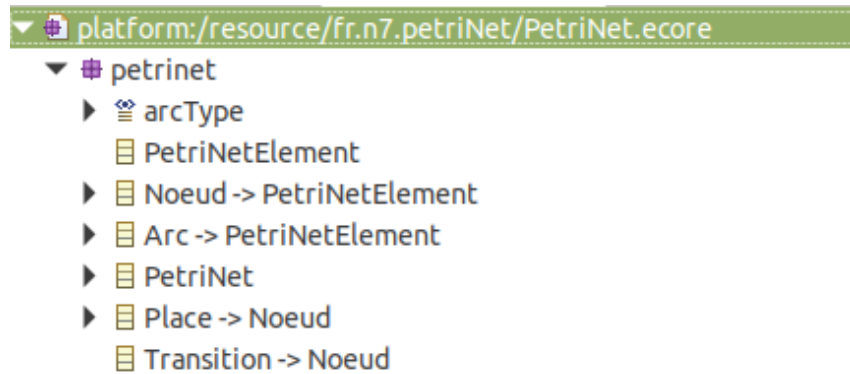


Figure 4: Méta-modèle PetriNet

On visualise le modèle PetriNet, et on obtient le méta-modèle dans la figure suivante la figure 3

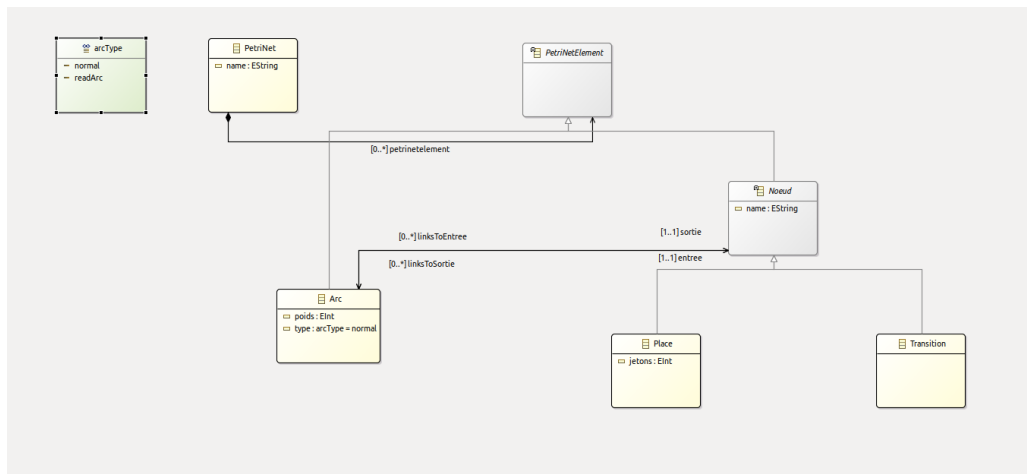


Figure 5: Méta-modèle PetriNet

## 4 Définitions des contraintes OCL (implantées en Java)

### 4.1 Contraintes pour SimplePDL

Les contraintes OCL pour SimplePDL sont définies comme suit :

- Les noms doivent respecter la règle suivante :

$$[A - Za - z][A - Za - z0 - 9]^*$$

- Deux activités et deux ressources ne doivent pas avoir le même nom.
- Une activité ne doit pas dépendre d'elle-même.
- La taille du nom d'une activité doit dépasser un caractère.
- Le nombre d'occurrences et la quantité des ressources doivent être positifs.

### 4.2 Contraintes pour PetriNet

Les contraintes OCL pour SimplePDL sont définies comme suit :

- Deux nœuds distincts ne peuvent pas porter le même nom.
- Un arc ne doit jamais connecter deux nœuds du même type.
- Le nombre de jetons dans une place doit toujours être positif ou nul.

## 5 Syntaxes concrètes de SimplePDL

### 5.1 Syntaxe concrète textuelle de SimplePDL (avec Xtext)

En utilisant l'outil Xtext, on définit une syntaxe textuelle concrète pour SimplePDL. Pour chaque élément du modèle, on spécifie une grammaire analysable en LL(k), permettant ainsi de générer le méta-modèle SimplePDL. Cette syntaxe textuelle, offre un moyen pratique et accessible de construire et de modifier des modèles conformes à ce méta-modèle. Xtext fournit un éditeur syntaxique avec des fonctionnalités avancées telles que la coloration syntaxique, l'auto-complétion, et la détection des erreurs.

Un fichier écrit avec Xtext est en fait un modèle conforme au métamodèle que Xtext génère, ainsi on peut le visualiser comme illustré dans la figure 7

```

process developpement {
  wd Conception utilise concepteur quantite 2, utilise machine quantite 2
  wd RedactionDoc utilise machine quantite 1, utilise redacteur quantite 1
  wd Programmation utilise developpeur quantite 2, utilise machine quantite 3
  wd RedactionTests utilise machine quantite 2, utilise testeur quantite 1

  ws f2f from Conception to RedactionDoc
  ws f2s from Conception to Programmation
  ws f2f from Programmation to RedactionTests
  ws f2f from RedactionDoc to RedactionTests

  note "Ceci est une note explicative sur le processus."

  ressource concepteur quantite 3
  ressource developpeur quantite 2
  ressource machine quantite 4
  ressource redacteur quantite 1
  ressource testeur quantite 2
}

```

Figure 6: Le modèle suivant la syntaxe textuelle

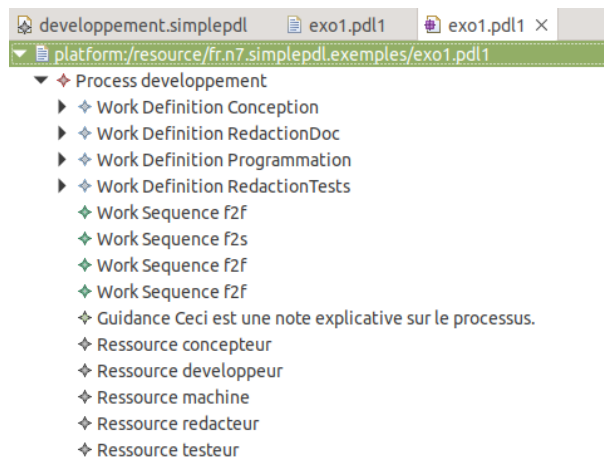


Figure 7: Visualisation du modèle avec l'éditeur Ecore

## 5.2 Syntaxe concrète graphique de SimplePDL (avec Siruis)

Les syntaxes graphiques sont essentielles pour rendre la visualisation des modèles plus intuitive et agréable. C'est pourquoi nous avons utilisé l'outil Sirius d'Eclipse, basé sur les technologies Eclipse Modeling (EMF), afin de générer un éditeur graphique à partir d'un modèle Ecore. Nous avons réussi à définir une syntaxe graphique pour le méta-modèle défini par Ecore, permettant de représenter graphiquement, pour un Process, les WorkDefinitions, les WorkSequences, les guidances ainsi que les Ressources, comme illustré par la figure ci-dessous.

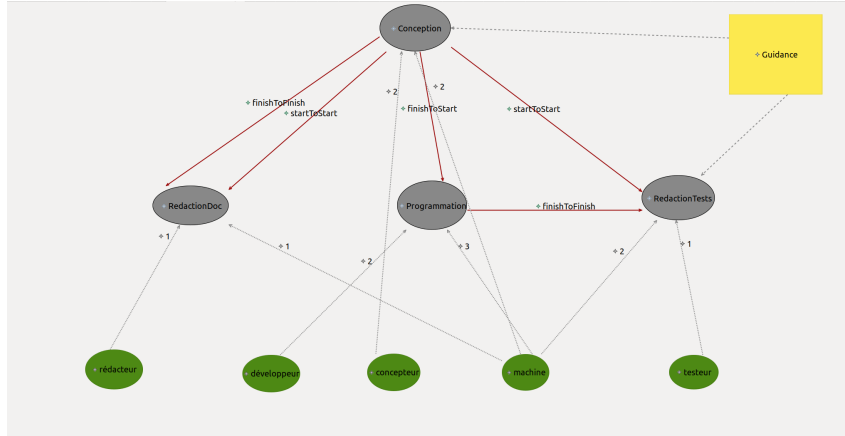


Figure 8: Syntaxe graphique du modèle développement

## 6 Définition de la transformation modèle à modèle (M2M)

Dans cette section, nous avons établi des transformations du modèle SimplePDL vers le réseau de Petri. Pour ce faire, nous avons utilisé deux langages : Java et ATL.

Le principe de transformation d'un modèle SimplePDL en Réseau de Pétri est le suivant :

- Un élément Process est converti en un élément PetriNet.
- Une WorkDefinition se transforme en quatre places : ready, started, running et finished, ainsi qu'en deux transitions : start et finish.

Pour tester ces transformations, nous avons utilisé le modèle suivant:

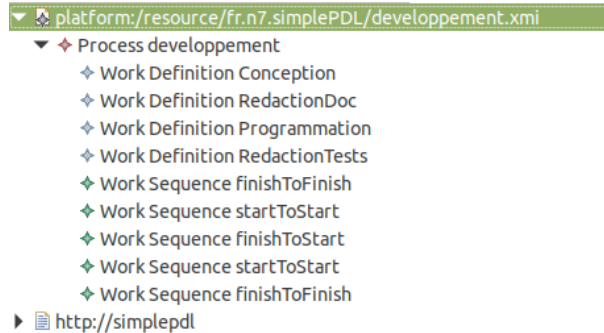


Figure 9: Exemple de modèle SimplePDL

### 6.1 Définition d'une transformation SimplePDL vers PetriNet en utilisant EMF/Java

En utilisant le langage Java, nous avons défini la transformation SimplePDL2PetriNet. Le résultat de la transformation est le suivant:



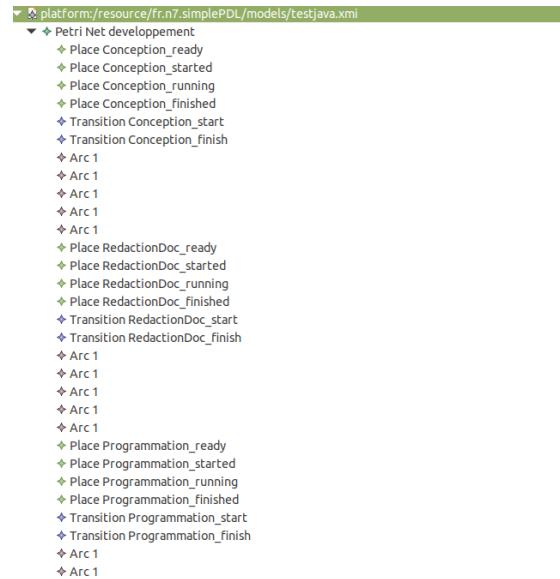


Figure 10: Résultat de la transformation avec EMF/Java

## 6.2 Définition d'une transformation SimplePDL vers PetriNet en utilisant ATL

En utilisant le langage ATL, nous avons défini la transformation SimplePDL2PetriNet. Le résultat de la transformation est le suivant:

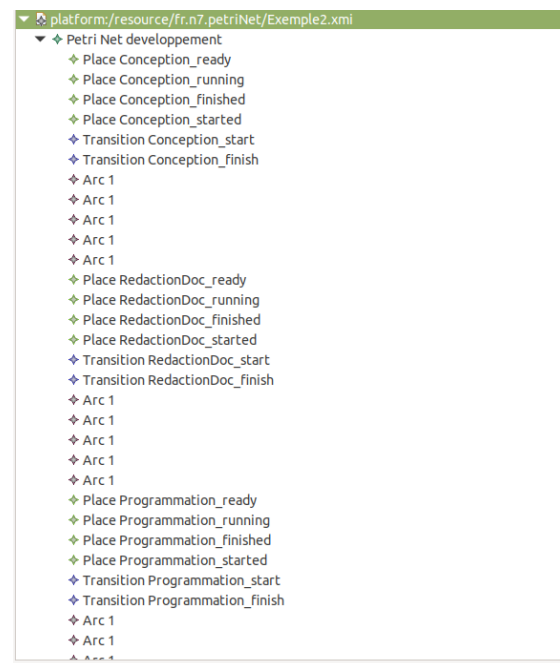


Figure 11: Résultat de la transformation avec EMF/Java

## 7 Définition de la transformation modèle à texte(M2T) en utilisant Acceleo

### 7.1 Définition d'une transformation PetriNet vers Tina

La transformation vers la syntaxe de Tina (.net) à partir du modèle PetriNet consiste à représenter chaque place par le mot-clé pl, suivi du nombre de jetons entre parenthèses. Les transitions sont également converties en tenant compte des poids associés et en vérifiant si les arcs sont de type readArc.

Le résultat de cette transformation appliquée au processus développement de la figure 11 est illustré dans la figure suivante:

```
1 net developpement
2 pl Conception_ready (1)
3 pl Conception_running (0)
4 pl Conception_finished (0)
5 pl Conception_started (0)
6 pl RedactionDoc_ready (1)
7 pl RedactionDoc_running (0)
8 pl RedactionDoc_finished (0)
9 pl RedactionDoc_started (0)
10 pl Programmation_ready (1)
11 pl Programmation_running (0)
12 pl Programmation_finished (0)
13 pl Programmation_started (0)
14 pl RedactionTests_ready (1)
15 pl RedactionTests_running (0)
16 pl RedactionTests_finished (0)
17 pl RedactionTests_started (0)
18 tr Conception_start Conception_ready -> Conception_started Conception_running
19 tr Conception_finish Conception_running -> Conception_finished
20 tr RedactionDoc_start RedactionDoc_ready Conception_started?1 -> RedactionDoc_started RedactionDoc_running
21 tr RedactionDoc_finish RedactionDoc_running Conception_finished?1 -> RedactionDoc_finished
22 tr Programmation_start Programmation_ready Conception_finished?1 -> Programmation_started Programmation_running
23 tr Programmation_finish Programmation_running -> Programmation_finished
24 tr RedactionTests_start RedactionTests_ready Conception_started?1 -> RedactionTests_started RedactionTests_running
25 tr RedactionTests_finish RedactionTests_running Programmation_finished?1 -> RedactionTests_finished
26
```

Figure 12: Modèle Développement avec syntaxe de Tina (.net)

le fichier (.net) peut être visualisé à travers Tina.

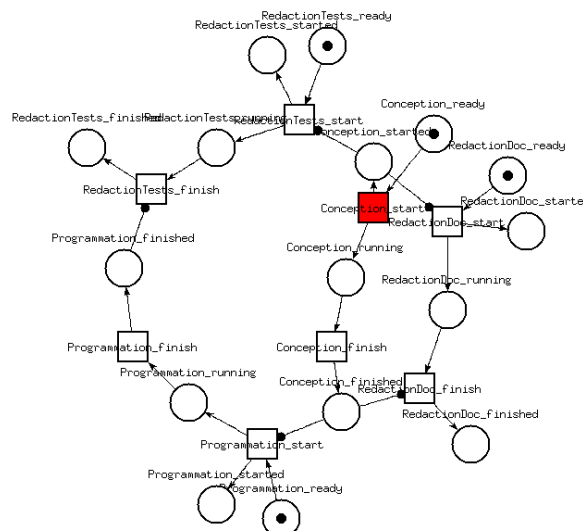


Figure 13: Syntaxe graphique de la transformation M2T

## 7.2 Définition d'une transformation SimplePDL vers LTL

Les propriétés LTL sont dérivées de SimplePDL pour être vérifiées à l'aide de l'outil Selt de Tina. Ces propriétés garantissent que les processus se terminent, qu'une activité ne peut pas se trouver simultanément dans plusieurs états, et qu'un processus démarré reste actif. L'outil Selt permet ensuite de vérifier si ces propriétés sont effectivement respectées dans le modèle de réseau de Petri.

Nous effectuons deux transformations vers LTL. La première transformation vise à vérifier la terminaison du processus, tandis que la seconde a pour objectif de s'assurer que le processus respecte les invariants. Ces transformations permettent ainsi de valider différents aspects du modèle et de garantir son bon fonctionnement.

Le résultat de ces deux transformations appliquée au processus développement de la figure 11 est illustré par les figures suivantes:

```
1
2 op finished = Conception_finished /\ RedactionDoc_finished /\ Programmation_finished /\ RedactionTests_finished;
3
4
5 [] (finished ==> dead);
6 [] <=> dead;
7 [] (dead ==> finished);
8 - <=> finished;
9
```

Figure 14: Les propriétés LTL permettant de vérifier la terminaison du processus développement



```
SimplePDL.ecore  PetriNet.ecore  developpement_finished.ltl  developpement.ltl x
1 <=> (Conception_finished /\ RedactionDoc_finished /\ Programmation_finished /\ RedactionTests_finished);
2
3 [] (Conception_ready + Conception_running + Conception_finished = 1);
4 [] (RedactionDoc_ready + RedactionDoc_running + RedactionDoc_finished = 1);
5 [] (Programmation_ready + Programmation_running + Programmation_finished = 1);
6 [] (RedactionTests_ready + RedactionTests_running + RedactionTests_finished = 1);
7
8 [] (Conception_started ==> [] (Conception_started));
9 [] (RedactionDoc_started ==> [] (RedactionDoc_started));
10 [] (Programmation_started ==> [] (Programmation_started));
11 [] (RedactionTests_started ==> [] (RedactionTests_started));
12
```

Figure 15: Les propriétés LTL correspondant aux invariants de SimplePDL pour valider la transformation écrite

## 8 Validation de la transformation SimplePDL2PetriNet

Pour faciliter la vérification de la transformation de SimplePDL en PetriNet, TINA propose un model-checker basé sur la logique temporelle LTL. Cette logique permet d'exprimer et de vérifier des propriétés essentielles de la transformation. Tout d'abord, nous avons généré les propriétés LTL pour vérifier les invariants de SimplePDL. Nous avons constaté que toutes les propriétés définies ont été validées avec succès.

Ensuite, nous avons généré les propriétés LTL pour vérifier la terminaison de processus.

```

aeh3860@hypotreme:~/eclipse-workspace-3a/fr/n7/simplePDL$ cd ..
aeh3860@hypotreme:~/eclipse-workspace-3a$ cd fr/n7/petrinet/totina
aeh3860@hypotreme:~/eclipse-workspace-3a/fr/n7/petrinet/totina$ /mnt/n7fs/ens/tp_pantel/tina/bin/selt -p -S developement.scn developement.ktz -prelude developement_finished.ltl
Selt version 3.4.4 -- 01/05/16 -- LAAS/CNRS
ktz loaded, 26 states, 47 transitions
0.005s

- source developement_finished.ltl;
source: cannot open developement_finished.ltl

- ^C
aeh3860@hypotreme:~/eclipse-workspace-3a/fr/n7/petrinet/totina$ /mnt/n7fs/ens/tp_pantel/tina/bin/selt -p -S developement.scn developement.ktz -prelude developement_finished.ltl
Selt version 3.4.4 -- 01/05/16 -- LAAS/CNRS
ktz loaded, 26 states, 47 transitions
0.001s

- source developement.ltl;
TRUE
TRUE
TRUE
TRUE
TRUE
TRUE
TRUE
TRUE
0.006s

```

Figure 16: Résultats de vérification des propriétés LTL correspondant aux invariants

```

aeh3860@hypotreme:~/eclipse-workspace-3a/fr/n7/petrinet/totina$ /mnt/n7fs/ens/tp_pantel/tina/bin/selt -p -S developement.scn developement.ktz -prelude developement_finished.ltl
Selt version 3.4.4 -- 01/05/16 -- LAAS/CNRS
ktz loaded, 26 states, 47 transitions
0.003s

- source developement_finished.ltl;
operator finished : prop
TRUE
TRUE
TRUE
FALSE
state 0: Conception_ready Programmation_ready RedactionDoc_ready RedactionTests_ready
->Conception_start->
state 1: Conception_running Conception_started Programmation_ready RedactionDoc_ready RedactionTests_ready
->Conception_finish->
state 2: Conception_finished Conception_started Programmation_ready RedactionDoc_ready RedactionTests_ready
->Programmation_start->
state 3: Conception_finished Conception_started Programmation_running Programmation_started RedactionDoc_ready RedactionTests_ready
->Programmation_finish->
state 4: Conception_finished Conception_started Programmation_finished Programmation_started RedactionDoc_ready RedactionTests_ready
->RedactionDoc_start->
state 5: Conception_finished Conception_started Programmation_finished Programmation_started RedactionDoc_running RedactionDoc_started RedactionTests_ready
->RedactionDoc_finish->
state 6: Conception_finished Conception_started Programmation_finished Programmation_started RedactionDoc_finished RedactionDoc_started RedactionTests_ready
->RedactionTests_start->
state 7: Conception_finished Conception_started Programmation_finished Programmation_started RedactionDoc_finished RedactionDoc_started RedactionTests_running RedactionTests_started
->RedactionTests_finish->
state 8: L_dead Conception_finished Conception_started Programmation_finished Programmation_started RedactionDoc_finished RedactionDoc_started RedactionTests_finished RedactionTests_started
->L_deadlock->
state 9: L_dead Conception_finished Conception_started Programmation_finished Programmation_started RedactionDoc_finished RedactionDoc_started RedactionTests_finished RedactionTests_started
[accepting all]
0.005s

```

Figure 17: Résultats de vérification des propriétés LTL correspondant à la terminaison

## 9 Conclusion

En conclusion, ce mini-projet nous a permis de développer une chaîne complète de vérification de modèles de processus, en utilisant différentes techniques d'ingénierie dirigée par les modèles (IDM). Nous avons conçu des métamodèles, défini des transformations et généré des propriétés LTL pour valider la terminaison des processus. Grâce à l'utilisation d'outils tels qu'EMF, Xtext, Sirius, et Aceleo, nous avons pu créer des éditeurs graphiques et textuels, facilitant ainsi la manipulation et la vérification des modèles. Ce projet a permis de renforcer notre compréhension des concepts de model-checking et de vérification de cohérence dans les processus métiers.